How to Setup Python on Your Workstation

It can be difficult to install a Python machine learning environment on some platforms. Python itself must be installed first and then there are many packages to install, and it can be confusing for beginners. In this tutorial, you will discover how to setup a Python machine learning development environment using Anaconda. After completing this tutorial, you will have a working Python environment to begin learning, practicing, and developing machine learning software.

A.1 Overview

In this chapter, we will cover the following steps:

1. Download Anaconda
2. Install Anaconda
3. Start and Update Anaconda
4. Install Deep Learning Libraries
5. Install Visual Studio Code environment

Note: The specific versions may differ as the software and libraries are updated frequently.

A.2 Download Anaconda

In this step, you will download the Anaconda Python package for your platform. Anaconda is a free and easy-to-use environment for scientific Python. These instructions are suitable for Windows, macOS, and Linux platforms. Windows are used below for demonstration, so you may see some Windows dialogs and file extensions.

1. Visit the Anaconda homepage `https://www.anaconda.com/`

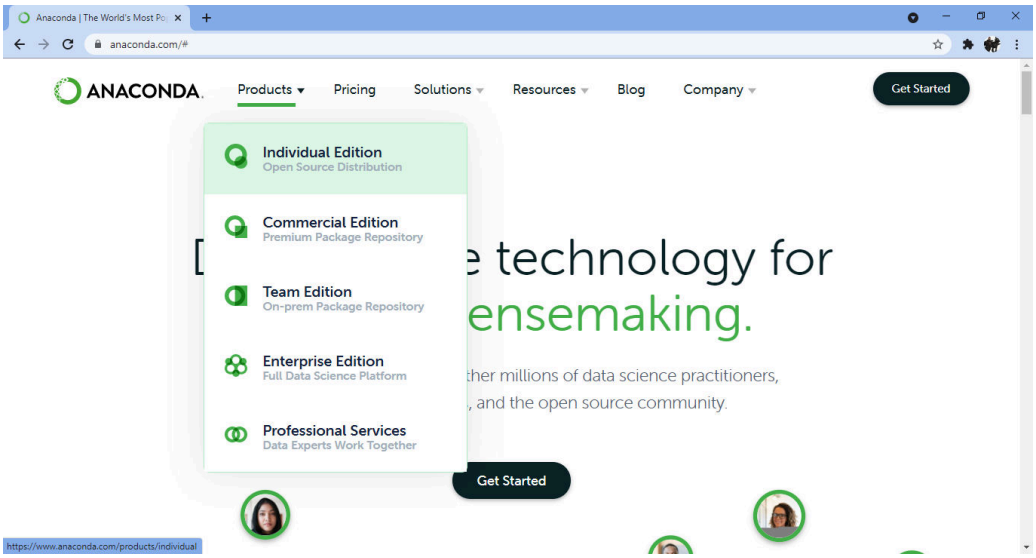

Figure A.1: Click "Products" and "Individual Edition"

2. Click "Products" from the menu and click "Individual Edition" to go to the download page `https://www.anaconda.com/products/individual-d/`.

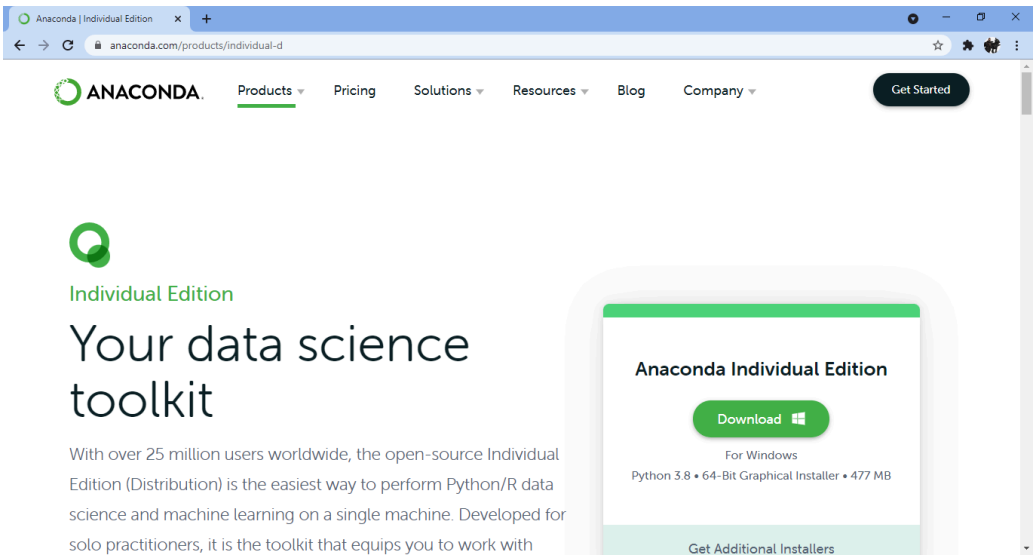

Figure A.2: Click Download

This will download the Anaconda Python package to your workstation. It will automatically give you the installer according to your OS (Windows, Linux, or MacOS). The file is about 480 MB. You should have a file with a name like:

```
Anaconda3-2021.05-Windows-x86_64.exe
```

A.3 Install Anaconda

In this step, you will install the Anaconda Python software on your system. This step assumes you have sufficient administrative privileges to install software on your system.

1. Double click the downloaded file.
2. Follow the installation wizard.

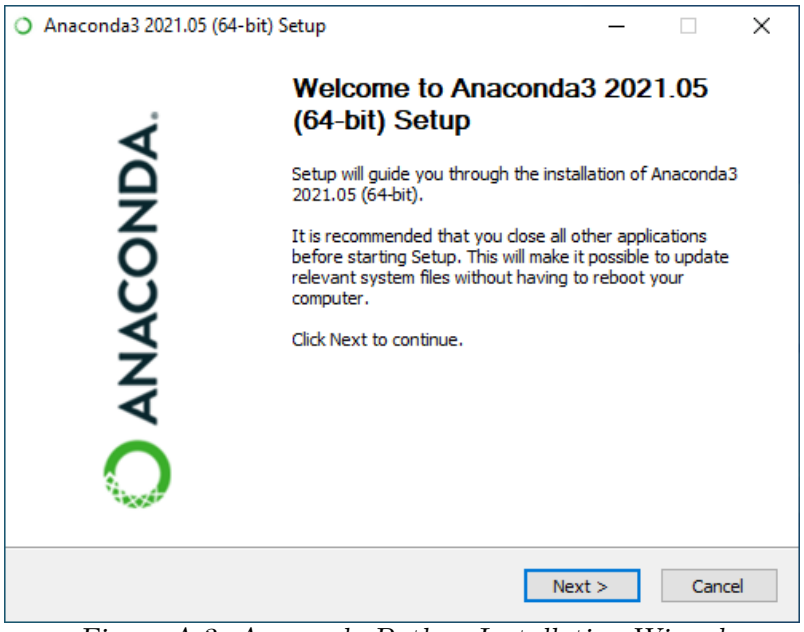

Figure A.3: Anaconda Python Installation Wizard

Installation is quick and painless. There should be no tricky questions or sticking points.

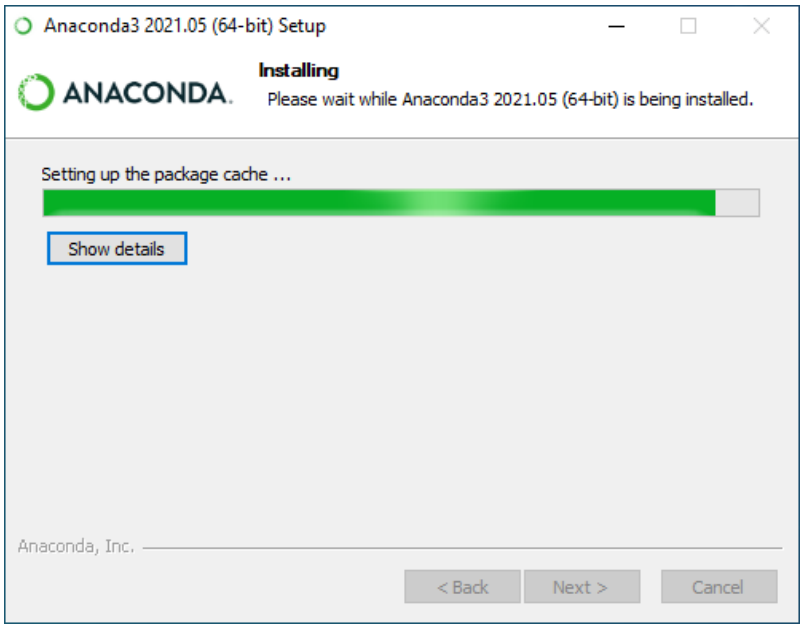

Figure A.4: Anaconda Python Installation Wizard Writing Files

The installation should take less than 10 minutes and take a bit more than 5 GB of space on your hard drive.

A.4 Start and Update Anaconda

In this step, you will confirm that your Anaconda Python environment is up to date. Anaconda comes with a suite of graphical tools called Anaconda Navigator. You can start Anaconda Navigator by opening it from your application launcher.

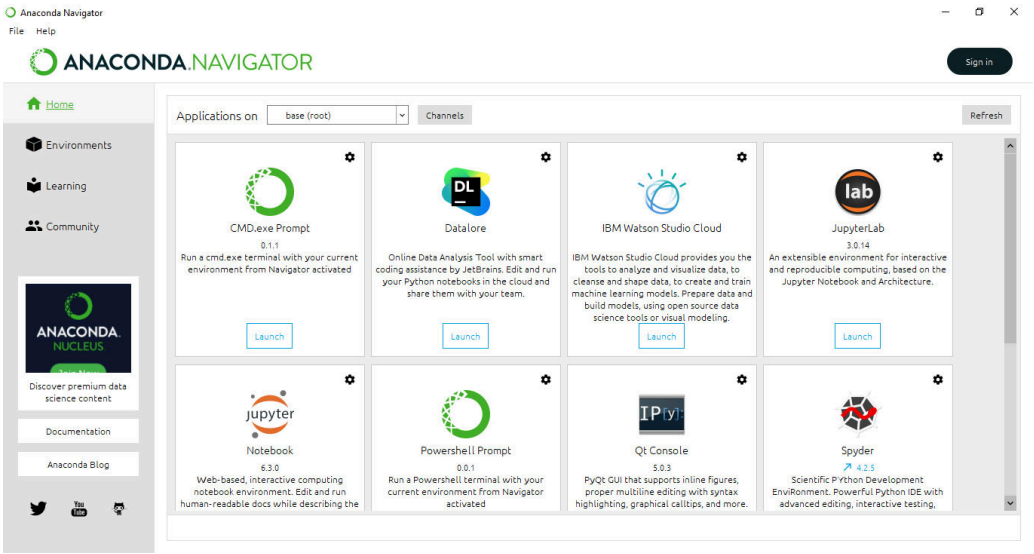

Figure A.5: Anaconda Navigator GUI

You can use the Anaconda Navigator and graphical development environments later; for now, it is recommended to start with the Anaconda command line environment called conda[1]. Conda is fast, simple, it's hard for error messages to hide, and you can quickly confirm your environment is installed and working correctly.

1. Open a terminal or CMD.exe prompt (command line window).
2. Confirm conda is installed correctly, by typing:

```
conda -V
```

You should see the following (or something similar):

```
conda 4.10.1
```

3. Confirm Python is installed correctly by typing:

```
python -V
```

[1] https://conda.pydata.org/docs/index.html

You should see the following (or something similar):

```
Python 3.8.8
```

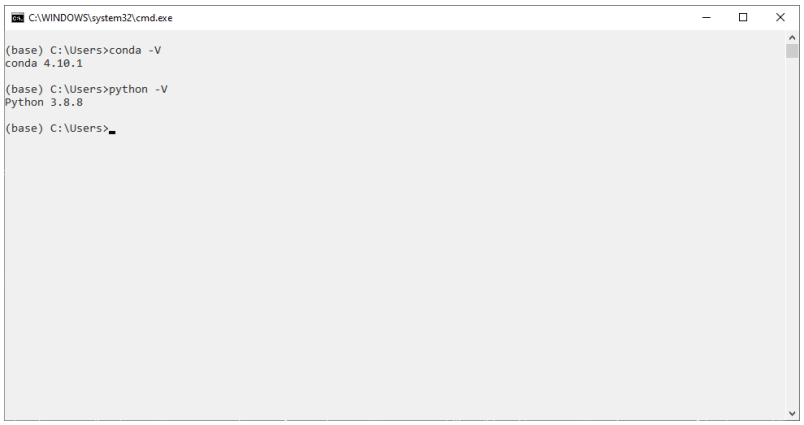

Figure A.6: Confirm Conda and Python are Installed

If the commands do not work or have an error, please check the documentation for help for your platform. See some of the resources in the "Further Reading" section.

4. This step is optional. You can make community supported packages available in conda by adding a "conda-forge" channel:

```
conda config --add channels conda-forge
```

5. Confirm your conda environment is up-to-date, type:

```
conda update conda
conda update anaconda
```

You may need to install some packages and confirm the updates.

6. Confirm your SciPy environment.

The script below will print the version number of the key SciPy libraries you require for machine learning development, specifically: SciPy, NumPy, Matplotlib, Pandas, Statsmodels, and Scikit-learn. You can type "`python`" and type the commands in directly. Alternatively, it recommended to open a text editor and copy-pasting the script into your editor.

```
# check library version numbers
# scipy
import scipy
print('scipy: %s' % scipy.__version__)
# numpy
import numpy
```

```
print('numpy: %s' % numpy.__version__)
# matplotlib
import matplotlib
print('matplotlib: %s' % matplotlib.__version__)
# pandas
import pandas
print('pandas: %s' % pandas.__version__)
# statsmodels
import statsmodels
print('statsmodels: %s' % statsmodels.__version__)
# scikit-learn
import sklearn
print('sklearn: %s' % sklearn.__version__)
```

Listing A.1: Code to check that key Python libraries are installed.

Save the script as a file with the name: `versions.py`. On the command line, change your directory to where you saved the script and type:

```
python versions.py
```

You should see output like the following:

```
scipy: 1.8.1
numpy: 1.23.1
matplotlib: 3.5.1
pandas: 1.4.3
statsmodels: 0.13.2
sklearn: 1.1.1
```

Output A.1: Sample output of thhe versions script

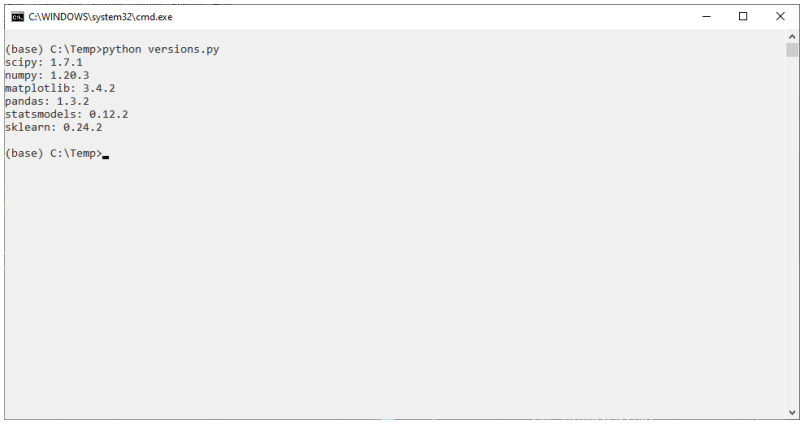

Figure A.7: Confirm Anaconda SciPy environment

A.5 Install Deep Learning Libraries

In this step, you will install Python libraries used for deep learning, specifically PyTorch:

1. Install the PyTorch and torchvision deep learning library by typing:

```
conda install pytorch torchvision
```

2. Alternatively, you may choose to install using `pip` and a specific version of PyTorch for your platform

```
pip install torch==2.0.0
```

3. Confirm your deep learning environment is installed and working correctly.

   Create a script that prints the version numbers of each library, as you did before for the SciPy environment.

```
# check deep learning version numbers
# pytorch
import torch
print('pytorch: %s' % torch.__version__)
# torchvision
import torchvision
print('torchvision: %s' % torchvision.__version__)
```

Listing A.2: Code to check that key deep learning libraries are installed.

   Save the script to a file `deep_versions.py`. Run the script by typing:

```
python deep_versions.py
```

Output A.2: Run script from the command line.

   You should see output like:

```
pytorch: 2.0.0
torchvision: 0.15.1
```

Output A.3: Sample output of the deep learning versions script

A.6 Install Visual Studio Code for Python

If you have installed Anaconda, you will have an IDE called Spyder installed. You can develop your Python project with Spyder. The other popular way of writing Python code nowadays is to use Visual Studio Code. It is a free programming environment from Microsoft. Visual Studio Code can do a lot of things via "extensions". Python extension is what you should get.

These instructions are suitable for Windows, macOS, and Linux platforms. Below, macOS is used to demonstrate, so you may see some Windows dialogs and file extensions.

1. Visit the Visual Studio Code homepage https://code.visualstudio.com

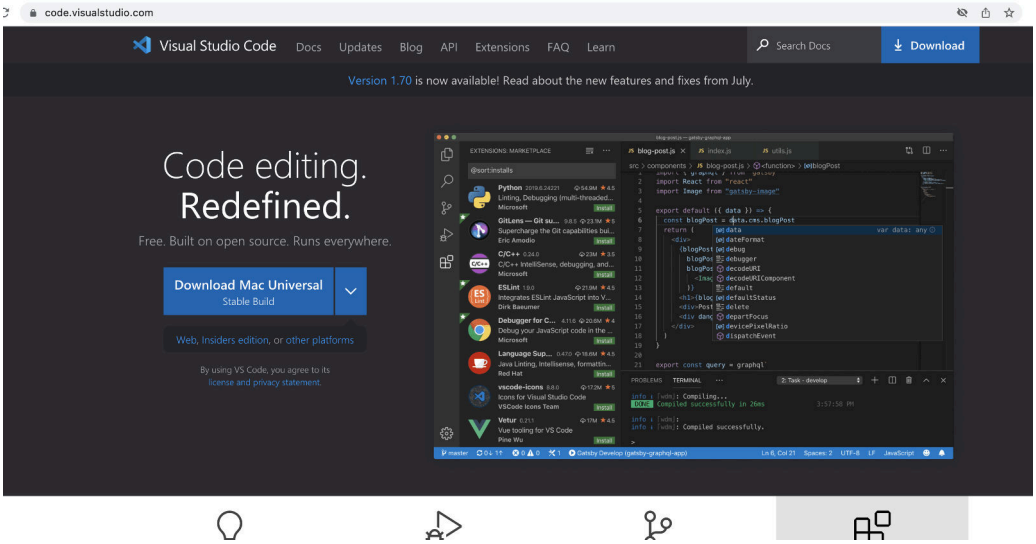

Figure A.8: Click "Products" and "Individual Edition"

2. Click the download button at the top toolbar or at the center of the screen to download a ZIP file (such as `VSCode-darwin-universal.zip`). It is around 200MB in size
3. Expand the ZIP you will find the application program. In macOS, you should move it into your `/Applications` folder

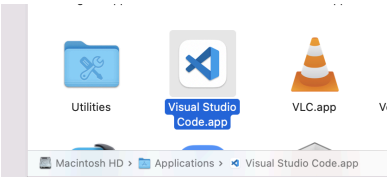

Figure A.9: Visual Studio Code in `/Applications` in macOS

4. Running Visual Studio Code will show you the main screen like the following:

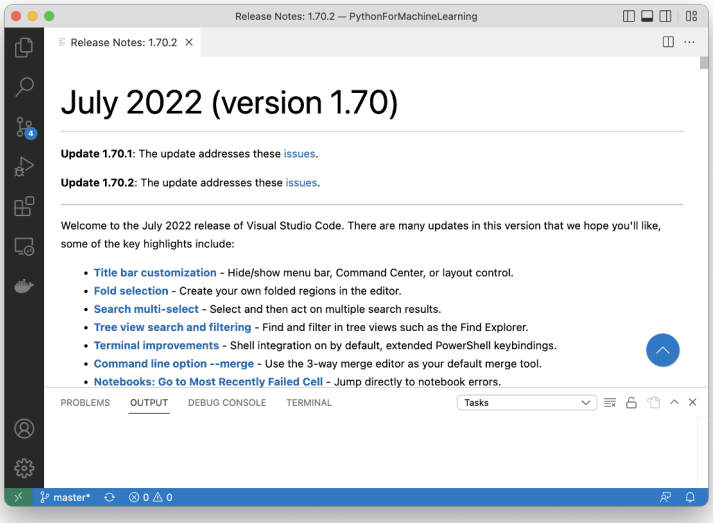

Figure A.10: Visual Studio Code main screen

You can click on the building block icon at left toolbar to open the extensions marketplace. Typing "`python`" on the search box will usually show the Microsoft-developed Python extension at top.

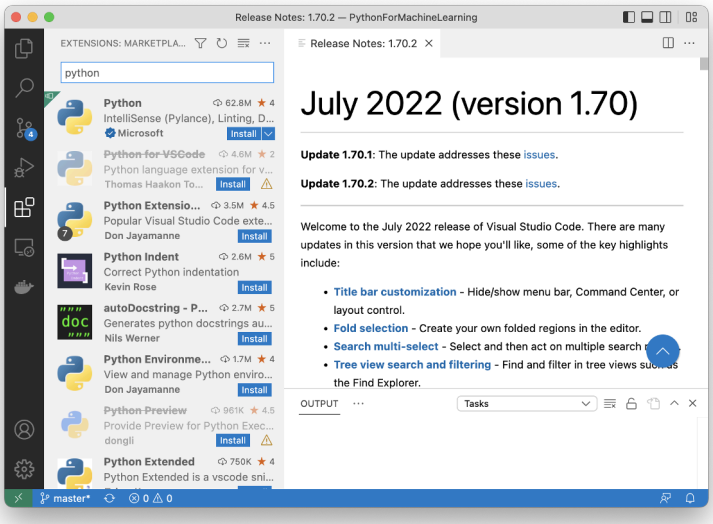

Figure A.11: Searching for Python extension in the extensions marketplace

5. Click on the "Install' button on the extension marketplace will get the extension installed. It should be very quick.

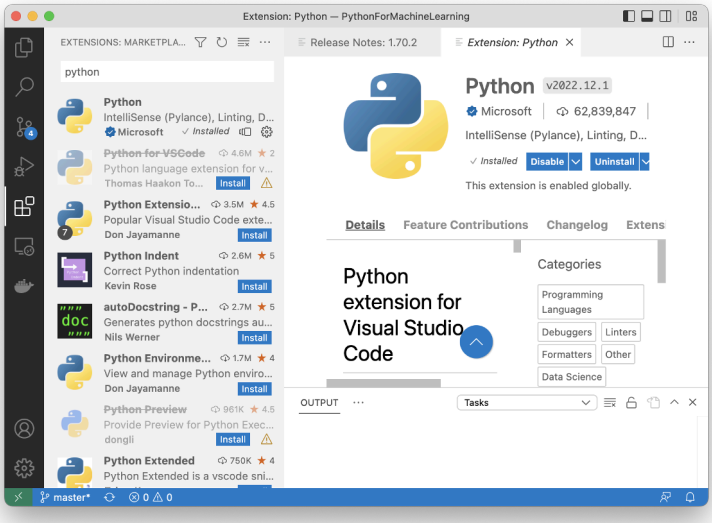

Figure A.12: Python extension installed in Visual Studio Code

Afterwards, you will find your Visual Studio Code can launch Python interpreter or Python debugger to run your program.

A.7 Further Reading

This section provides resources if you want to know more about Anaconda.

- Anaconda Documentation
  https://docs.continuum.io/
- Anaconda Documentation: Installation
  https://docs.continuum.io/anaconda/install
- Anaconda Navigator
  https://docs.continuum.io/anaconda/navigator.html
- The conda command line tool
  https://conda.pydata.org/docs/index.html
- Using conda
  https://conda.pydata.org/docs/using/
- conda-forge
  https://conda-forge.org//docs/using/
- Instructions for installing TensorFlow in Anaconda
  https://docs.anaconda.com/anaconda/user-guide/tasks/tensorflow/

A.8 Summary

Congratulations, you now have a working Python development environment for machine learning. You can now learn and practice machine learning on your workstation.

How to Setup Amazon EC2 for Deep Learning on GPUs

B

Large deep learning models require a lot of compute time to run. You can run them on your CPU but it can take hours or days to get a result. If you have access to a GPU on your desktop, you can drastically speed up the training time of your deep learning models. In this project you will discover how you can get access to GPUs to speed up the training of your deep learning models by using the Amazon Web Service (AWS) infrastructure. For less than a dollar per hour and often a lot cheaper you can use this service from your workstation or laptop. After working through this project you will know:

- How to create an account and log-in to Amazon Web Service.
- How to launch a server instance for deep learning.
- How to configure a server instance for faster deep learning on the GPU.

Let's get started.

B.1 Overview

The process is quite simple because most of the work has already been done for us. Below is an overview of the process.

- Setup Your AWS Account.
- Launch Your Server Instance.
- Login and Run Your Code.
- Close Your Server Instance.

Note: It costs money to use a virtual server instance on Amazon. The cost is low for model development (e.g., less than one US dollar per hour), which is why this is so attractive, but it is not free. The server instance runs Linux. It is desirable although not required that you know how to navigate Linux or a Unix-like environment. We're just running our Python scripts, so no advanced skills are needed.

Note: The specific versions may differ as the software and libraries are updated frequently.

B.2 Setup Your AWS Account

You need an account on Amazon Web Services (`https://aws.amazon.com`).

1. Go to `https://aws.amazon.com`. You should click "Sign In" at the toolbar

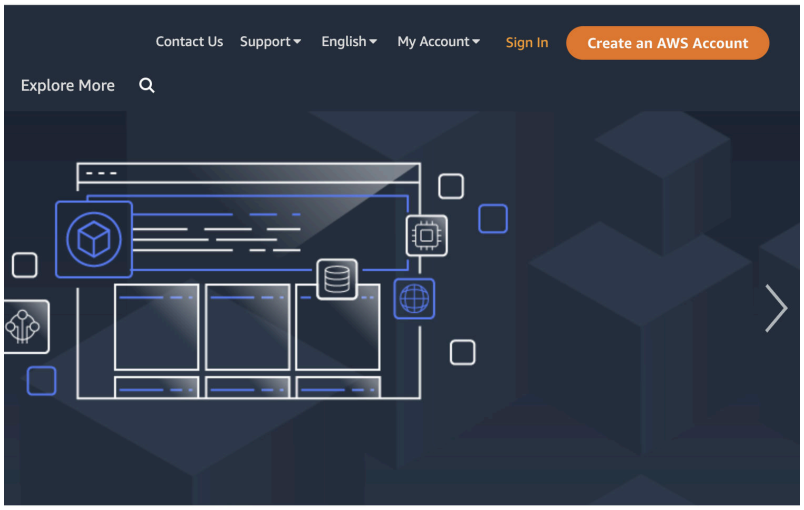

Figure B.1: AWS Sign In Button

2. This will bring you to the following screen. You can enter your login credentials, or you can sign up an account if you didn't have one yet.

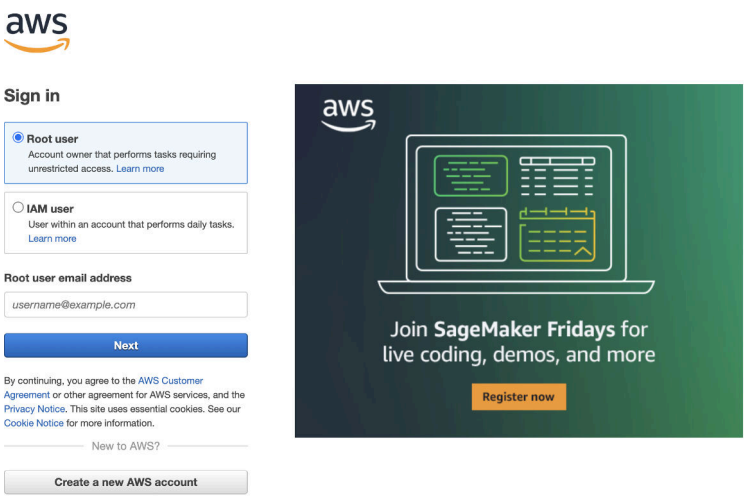

Figure B.2: Login screen for AWS console

Once you have an account you can log into the Amazon Web Services console. You will see a range of different services that you can access.

B.3 Launch Your Server Instance

Now that you have an AWS account, you want to launch an EC2 virtual server instance on which you can run Keras. Launching an instance is as easy as selecting the image to load and starting the virtual server. Thankfully there is already an image available that has almost everything we need it is called the "*Deep Learning AMI GPU PyTorch 1.13.1 (Ubuntu 20.04) 20230315*" and was created and is maintained by Amazon. Let's launch it as an instance.

1. Login to your AWS console if you have not already.
   `https://console.aws.amazon.com/console/home`
2. At the toolbar, you can select your server location. The server will cost differently in different region. Then click on EC2 for launching a new virtual server.

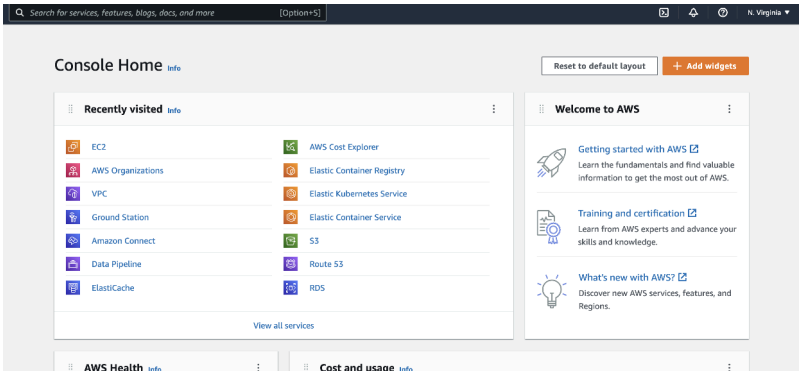

Figure B.3: AWS console

3. Click the *Launch instance* button.

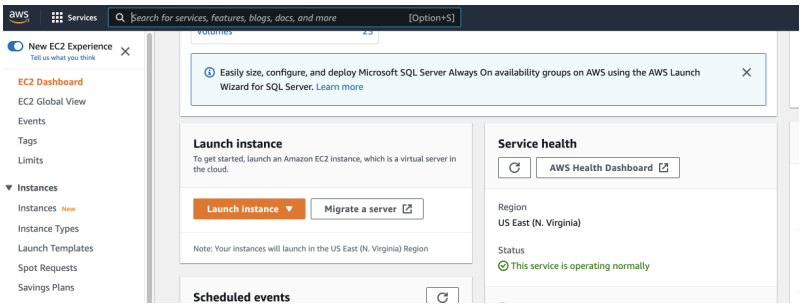

Figure B.4: Launch an EC2 instance

4. Give a name to your instance. Then click on *Browse more AMIs* and search with the keyword "deep learning". An AMI is an Amazon Machine Image. It is a frozen instance of a server that you can select and instantiate on a new virtual server.

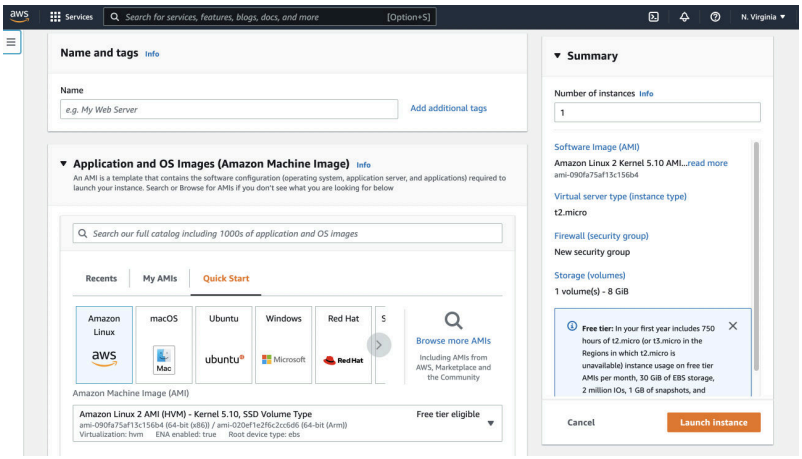

Figure B.5: Searching for an AMI

5. We will select the one with PyTorch pre-installed.

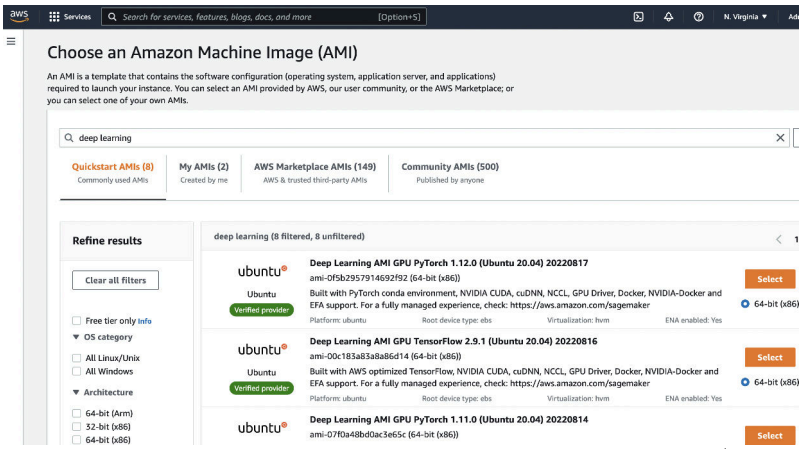

Figure B.6: Selecting "Deep Learning AMI GPU PyTorch 1.13.1 (Ubuntu 20.04)"

6. Now you need to select the hardware on which to run the image. The drop down box will show all the available hardware that are compatible to your AMI. The instance types begin with a "`g`" comes with GPU and those begin with a "`p`" includes a Tesla core GPU, which are much faster. In below, `g4dn.xlarge` is selected but you can pick one with suitable amount of memory, disk size, and cost for your project.

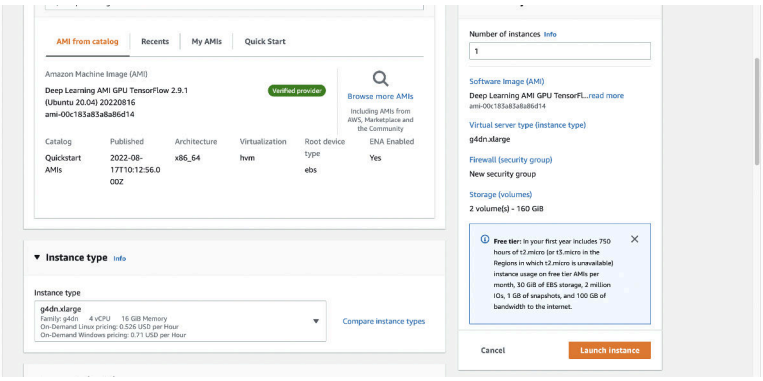

Figure B.7: Selecting instance type `g4ad.xlarge` for the new server

You can learn more about EC2 instance types at `https://aws.amazon.com/ec2/instance-types/`.

7. You need to select a key pair for your machine. It is used to login to the server via SSH using key-based authentication. You may select an existing one, or create a new key pair.

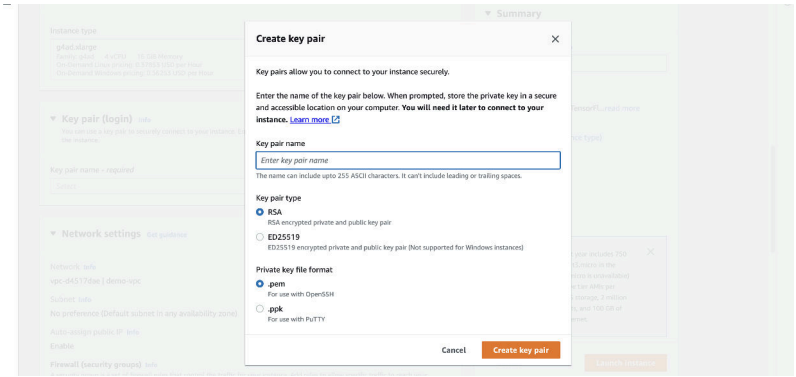

Figure B.8: Creating a new key pair

Depends on the program you use as your SSH client, you may select to download your key between PEM format or PPK format. Keep this key file safe. You will not be able to get it again after this step.

8. As a final check, you should see your instance with "Auto-assign public IP" enabled and "Allow SSH traffic from" set to "Anywhere" so we can access to the instance after launch. Then you can click *Launch* to create your instance.

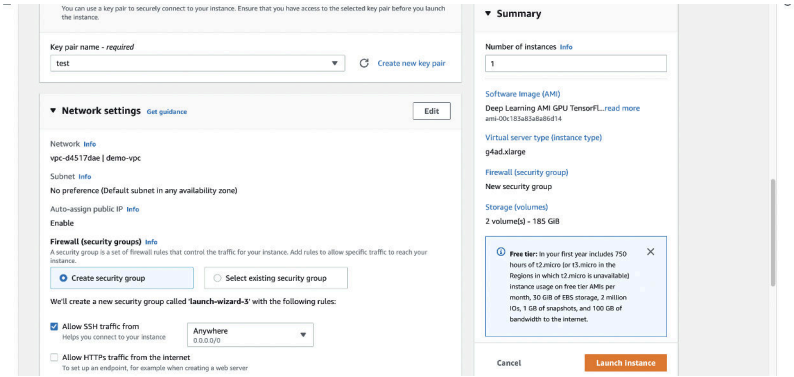

Figure B.9: Launch a new instance

Your server is now running and ready for you to log in.

B.4 Login, Configure and Run

Now that you have launched your server instance, it is time to log in and start using it.

1. Open a Terminal and change directory to where you downloaded your key pair.

2. If you have not already done so, restrict the access permissions on your key pair file. This is required as part of the SSH access to your server. For example, open a terminal on your workstation and type:

```
cd Downloads
chmod 600 my-aws-key.pem
```

Listing B.1: Change permissions of your key file

3. Click *Instances* in your Amazon EC2 console if you have not done so already. And find and select the instance that you just created.

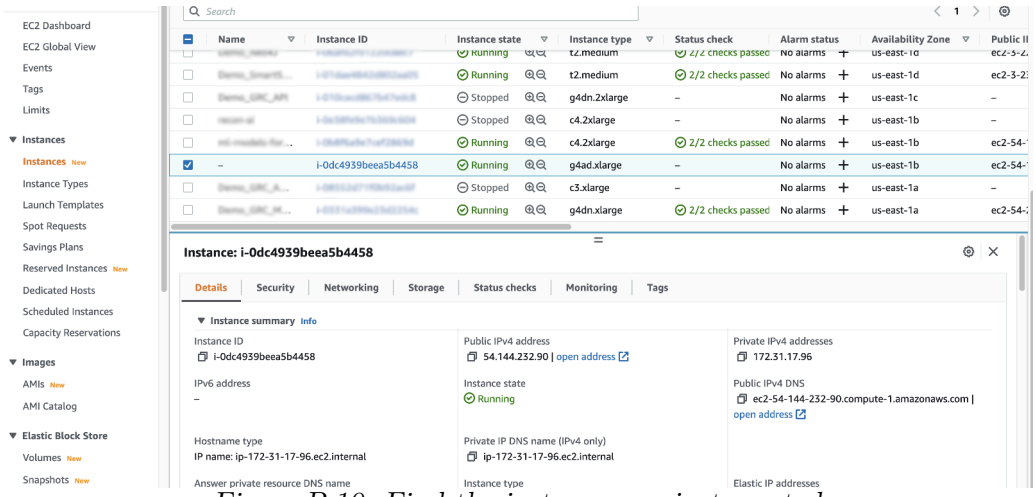

Figure B.10: Find the instance you just created

4. Copy the *Public IPv4 address* (at first row of the instance detail table) to your clipboard. In this example my IP address is `54.144.232.90`. **Do not use this IP address, it will not work as your server IP address will be different**.

5. Open a Terminal and change directory to where you downloaded your key pair. Login to your server using SSH, for example:

```
ssh -i my-aws-key.pem ubuntu@54.144.232.90
```

Listing B.2: Log-in To Your AWS Instance.

   If prompted, type `yes` and press enter.

   In the SSH command, we use the syntax `username@address` to specify the server address and the login name. The login name depends on the AMI you used. If you used Ubuntu AMI, it is `ubuntu`. But if you used Amazon Linux AMI, it would be `ec2-user`. You can see these details at https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/connection-prereqs.html

6. If your SSH session is established, you should see the login screen as follows:

```
=============================================================================
       __|  __|_  )
       _|  (     /   Deep Learning AMI GPU PyTorch 1.13.1 (Ubuntu 20.04)
      ___|\___|___|
=============================================================================

Welcome to Ubuntu 20.04.4 LTS (GNU/Linux 5.15.0-1017-aws x86_64v)


PyTorch 1.13.1 and utility libraries are installed in /usr/bin/python3.9.

To access them, use /usr/bin/python3.9.

NVIDIA driver version: 510.47.03
CUDA version: 11.2

AWS Deep Learning AMI Homepage: https://aws.amazon.com/machine-learning/amis/
Release Notes: https://docs.aws.amazon.com/dlami/latest/devguide/appendix-ami-release-notes.html
Support: https://forums.aws.amazon.com/forum.jspa?forumID=263
For a fully managed experience, check out Amazon SageMaker at https://aws.amazon.com/sagemaker
Security scan reports for python packages are located at: /opt/aws/dlami/info/
=============================================================================

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Mon Aug 22 20:44:29 UTC 2022


0 updates can be applied immediately.



The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the

individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-172-31-17-96:~$
```

Figure B.11: Login screen for your AWS server

You are now logged into your server. You are now free to run your code.

Depends on the AMI, you may need to launch Python differently. In this particular AMI, there are Python 3.8 and 3.9 installed but all deep learning libraries are available on Python 3.9 only. You need to use `python3.9` command instead of `python` or `python3` for your projects.

B.5 Build and Run Models on AWS

This section offers some tips for running your code on AWS.

B.5.1 Copy Scripts and Data to AWS

You can get started quickly by copying your files to your running AWS instance. For example, you can copy the examples provided with this book to your AWS instance using the `scp` command as follows:

```
scp -i my-aws-key.pem -r src ubuntu@54.144.232.90:~/
```

Listing B.3: Example for copying sample code to AWS

This will copy the entire `src/` directory to your home directory on your AWS instance. You can easily adapt this example to get your larger datasets from your workstation onto your AWS instance. Note that Amazon may impose charges for moving very large amounts of data in and out of your AWS instance. Refer to Amazon documentation for relevant charges.

B.5.2 Run Models on AWS

You can run your scripts on your AWS instance as per normal:

```
python3.9 filename.py
```

Listing B.4: Example of running a Python script on AWS

In other instances, you may use another command for the Python interpreter, such as `python3`. You are using AWS to create large neural network models that may take hours or days to train. As such, it is a better idea to run your scripts as a background job. This allows you to close your terminal and your workstation while your AWS instance continues to run your script. You can easily run your script as a background process as follows:

```
nohup /path/to/script >/path/to/script.log 2>&1 < /dev/null &
```

Listing B.5: Run script as a background process.

You can then check the status and results in your `script.log` file later.

B.6 Close Your EC2 Instance

When you are finished with your work you must terminate your instance. Remember you are charged by the amount of time that you use the instance. It is cheap, but you do not want to leave an instance on if you are not using it.

1. Log out of your instance at the terminal, for example you can type:

```
exit
```

Listing B.6: Command to log-out of server instance

2. Log in to your AWS account with your web browser.
3. Click EC2.
4. Click *Instances* from the left-hand side menu, then find and select your instance.
5. At the toolbar, click *Instance state* and select *Terminate instance*. Confirm that you want to terminate your running instance.

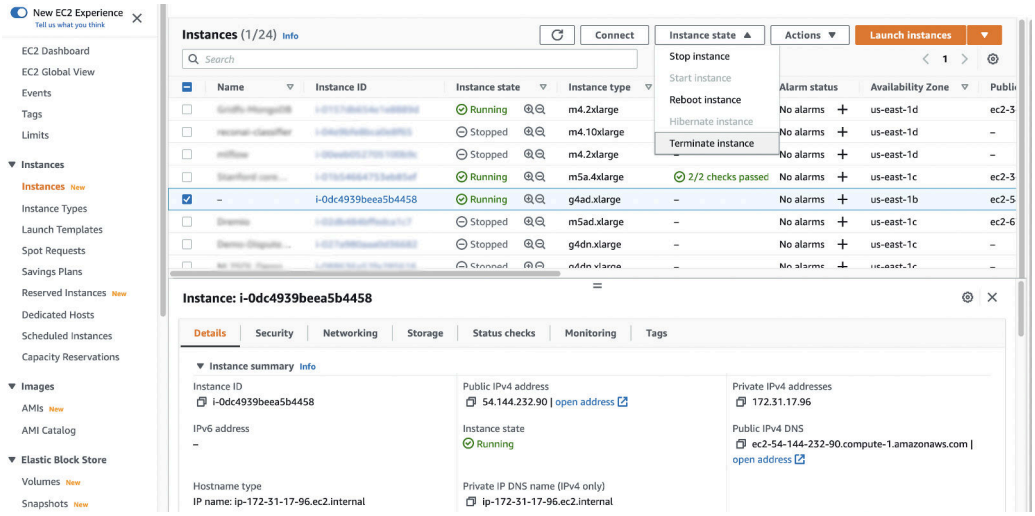

Figure B.12: Terminate an instance

There is a *Stop instance* option. That will only stop your instance from running but still occupies the resources such as storage and IP address. Hence terminate is how you stop AWS from billing you for the server.

It may take a number of seconds for the instance to close and to be removed from your list of instances.

B.7 Tips and Tricks for Using Keras on AWS

Below are some tips and tricks for getting the most out of using Keras on AWS instances.

- ▷ **Design a suite of experiments to run beforehand**. Experiments can take a long time to run and you are paying for the time you use. Make time to design a batch of experiments to run on AWS. Put each in a separate file and call them in turn from another script. This will allow you to answer multiple questions from one long run, perhaps overnight.
- ▷ **Always close your instance at the end of your experiments**. You do not want to be surprised with a very large AWS bill.
- ▷ **Try spot instances for a cheaper but less reliable option**. Amazon sell unused time on their hardware at a much cheaper price, but at the cost of potentially having your instance closed at any second. If you are learning or your experiments are not critical, this might be an ideal option for you. You can access spot instances from the *Spot Instance* option on the left hand side menu in your EC2 web console.

B.8 Further Reading

Below is a list of resources to learn more about AWS and developing deep learning models in the cloud.

- An introduction to Amazon Elastic Compute Cloud (EC2).
  http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html
- An introduction to Amazon Machine Images (AMI).
  http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html
- AWS instance types.
  https://aws.amazon.com/ec2/instance-types/

B.9 Summary

In this lesson you discovered how you can develop and evaluate your large deep learning models in Keras using GPUs on the Amazon Web Service. You learned:

- Amazon Web Services with their Elastic Compute Cloud offers an affordable way to run large deep learning models on GPU hardware.
- How to setup and launch an EC2 server for deep learning experiments.
- How to update the Keras version on the server and confirm that the system is working correctly.
- How to run Keras experiments on AWS instances in batch as background tasks.